**REKAYASA PERANGKAT LUNAK · PRACTICUM MODULE**

# Software Quality Assurance & Software Metrics

*Language-Agnostic Lab Exercises (Pseudocode-Based)*

Informatics Engineering Program · Academic Year 2026/2027 · International Class

# MODUL PRAKTIKUM

## MATA KULIAH REKAYASA PERANGKAT LUNAK

Disusun oleh:

**[ Dila Umnia Soraya, S.Pd., M.Pd. ]**

**[Gres Dyah Kusuma Ningrum, S.Pd., M.Pd.]**

**[ Fajar Pramudyana Ananta Purba ]**

**[ Aulia Elvira Aditama ]**

**[ Muhammad Avid Maulana ]**

Asisten Dosen Mata Kuliah Rekayasa Perangkat Lunak

*TAHUN AKADEMIK 2026/2027*

# General Instructions

## Learning Outcomes

By completing this practicum module, students will be able to:

1. Design black-box test cases using Equivalence Partitioning (EP) and Boundary Value Analysis (BVA).
2. Conduct a formal technical review / code inspection to identify defects in a given algorithm.
3. Evaluate a real software product against the ISO/IEC 25010 quality characteristics.
4. Map testing levels and testing types to a given system scenario.
5. Manually calculate Function Points, Cyclomatic Complexity, and Halstead metrics from a specification or pseudocode.
6. Calculate Defect Density, MTBF, MTTR, Defect Removal Efficiency, and the DORA Four Keys from case study data.
7. Critically analyze how metrics can be gamed (Goodhart's Law) and propose balanced alternatives.

## Format & Tools

This module is language-agnostic: all exercises use pseudocode and case-study data, so no specific programming language or IDE is required.

- Pen and paper, or a word processor, to complete the worksheets
- A calculator or spreadsheet application (e.g., Excel / Google Sheets) for numerical calculations
- A basic understanding of logarithms ($\log_2$) for the Halstead Volume calculation

## Suggested Time Allocation

| Practicum | Activities | Suggested Duration |
|---|---|---|
| Practicum 1 – SQA | Activities 1.1 – 1.4 (1.5 optional) | 100–150 minutes |
| Practicum 2 – Metrics | Activities 2.1 – 2.6 | 100–150 minutes |

## Submission

Complete all worksheets in this document (or a copy of it) and submit as a single file. Show your working for all calculations partial credit is given for correct method even if the final numeric answer contains an error.

# Practicum 1: Software Quality Assurance

This practicum reinforces Session 1 concepts: black-box test design, formal review, quality models, and testing levels/types.

## Activity 1.1 Black-Box Test Case Design (EP & BVA)

Recap: Equivalence Partitioning (EP) divides inputs into classes expected to be handled the same way; Boundary Value Analysis (BVA) targets the edges of those classes, where defects most often hide.

### Case Specification: Grade Classifier

Design a test suite for a function ClassifyGrade(score) with the following specification:

| Condition | Output |
|---|---|
| **90 ≤ score ≤ 100** | "A" |
| **75 ≤ score < 90** | "B" |
| **60 ≤ score < 75** | "C" |
| **0 ≤ score < 60** | "D" |
| **score < 0 OR score > 100** | "Invalid Input" |

**TASK Identify all equivalence classes (valid and invalid) and all boundary values, then design at least 10 test cases covering every class and boundary. Fill in the worksheet below.**

| Test ID | Input (score) | Type (EP / BVA) | Expected Output |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

| Test ID | Input (score) | Type (EP / BVA) | Expected Output |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

## Activity 1.2 Formal Technical Review / Code Inspection

Recap: A formal technical review (e.g., Fagan Inspection) is a structured, static examination of an artifact performed without executing it to find defects as early as possible.

### Artifact Under Review: CalculateAverage

The pseudocode below is intended to calculate the average of all non-negative numbers in an array, excluding negative values as outliers.

```
FUNCTION CalculateAverage(numbers[], size)
1:    sum = 0
2:    count = 0
3:    FOR i = 0 TO size
4:        IF numbers[i] > 0
5:            sum = sum + numbers[i]
6:            count = count + 1
7:        END IF
8:    END FOR
9:    IF count = 0
10:       RETURN 0
11:   END IF
12:   average = sum / count
13:  RETURN average
END FUNCTION
```

**TASK Conduct an individual walkthrough of the pseudocode above. Find at least 4 defects (logic errors, edge-case gaps, or poor practices) and log them below.**

| Defect ID | Line(s) | Description | Severity (Major/Minor) | Suggested Fix |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

## Activity 1.3 ISO/IEC 25010 Quality Evaluation

**TASK Choose one mobile or web application you use regularly. Score it from 1 (very poor) to 5 (excellent) on each ISO/IEC 25010 characteristic, with a concrete justification for every score.**

| Characteristic | Score (1–5) | Justification |
|---|---|---|
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

> ***Grading Tip:*** *Award full marks only when the justification cites a specific, concrete example from the chosen app not a generic statement.*

## Activity 1.4 Testing Level & Type Mapping

### Case: Campus Library Book Loan System

The system has four core modules: (1) Search Catalog, (2) Borrow Book, (3) Return Book & Pay Fine, (4) Generate Reports.

**TASK For each testing level below, write one concrete test scenario for this system. Then list which testing types (functional/non-functional, regression, etc.) would be most relevant and explain why.**

| Testing Level | Concrete Test Scenario |
|---|---|
| | |
| | |
| | |
| | |

Relevant testing types and justification:

| Testing Type | Why It Applies to This System |
|---|---|
| | |
| | |
| | |

## Activity 1.5 (Bonus) Continuous Testing Pipeline Design

**TASK Sketch a CI/CD pipeline for the Library System with at least 4 stages. For each stage, state what kind of test runs there and what happens if that quality gate fails.**

| Stage | Test(s) Performed | Action if Gate Fails |
|---|---|---|
| | | |

| Stage | Test(s) Performed | Action if Gate Fails |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

## Practicum 1 Deliverables Checklist

- ☐ Activity 1.1 worksheet completed (≥ 10 test cases)
- ☐ Activity 1.2 worksheet completed (≥ 4 defects logged)
- ☐ Activity 1.3 worksheet completed (all 8 characteristics scored & justified)
- ☐ Activity 1.4 worksheet completed (4 levels + testing types)
- ☐ Activity 1.5 completed (optional / bonus credit)

# Practicum 2: Software Metrics

This practicum reinforces Session 2 concepts: size, complexity, quality, and delivery metrics calculated entirely by hand or with a spreadsheet.

## Activity 2.1 Function Point Calculation

### Case: Student Attendance System

The system includes the following functions. Their complexity ratings have already been determined for you (in a real project, this requires a more detailed Data-Element / File-Type analysis, which is beyond the scope of this exercise).

| # | Function | Type | Complexity |
|---|---|---|---|
| 1 | Login | External Input (EI) | Average |
| 2 | Add Student Record | External Input (EI) | Average |
| 3 | Mark Attendance | External Input (EI) | Low |
| 4 | Generate Attendance Report | External Output (EO) | High |
| 5 | Search Student by Name | External Inquiry (EQ) | Low |
| 6 | Student Master File | Internal Logical File (ILF) | Average |
| 7 | Course Master File | Internal Logical File (ILF) | Low |
| 8 | External Payroll Interface | External Interface File (EIF) | Average |

Complexity weight table (simplified IFPUG-style):

| Type | Low | Average | High |
|---|---|---|---|
| External Input (EI) | 3 | 4 | 6 |
| External Output (EO) | 4 | 5 | 7 |
| External Inquiry (EQ) | 3 | 4 | 6 |
| Internal Logical File (ILF) | 7 | 10 | 15 |
| External Interface File (EIF) | 5 | 7 | 10 |

**TASK Using the tables above, assign the correct weight to each of the 8 functions and calculate the total Unadjusted Function Points (UFP).**

| # | Function | Weight | Running Total |
|---|---|---|---|
| | | | |
| | | | |

| # | Function | Weight | Running Total |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

## Activity 2.2 Cyclomatic Complexity Calculation

For each pseudocode snippet below: (a) identify every decision point, (b) calculate V(G) using V(G) = P + 1, and (c) classify the risk level using the interpretation table from the lecture (1–10 Simple, 11–20 Moderate, 21–50 Complex, >50 Untestable).

### Snippet A

```
FUNCTION CheckEven(number)
1: IF number MOD 2 = 0
2:     PRINT "Even"
3: ELSE
4:     PRINT "Odd"
5: END IF
END FUNCTION
```

### Snippet B

```
FUNCTION ClassifyScores(scores[], size)
1: FOR i = 0 TO size-1
2:     IF scores[i] >= 90
3:         grade = "A"
4:     ELSE IF scores[i] >= 75
5:         grade = "B"
6:     ELSE
7:         grade = "C"
8:     END IF
9:     IF grade = "A"
10:        countA = countA + 1
11:    END IF
```

```
12: END FOR
END FUNCTION
```

**Snippet C**

```
FUNCTION ValidateOrder(items[], size, customerType)
1:  IF size = 0
2:      RETURN "Empty order"
3:  END IF
4:  FOR i = 0 TO size-1
5:      IF items[i].quantity <= 0
6:          RETURN "Invalid quantity"
7:      END IF
8:      IF items[i].price < 0
9:          RETURN "Invalid price"
10:     END IF
11:     IF items[i].stock < items[i].quantity
12:         RETURN "Out of stock"
13:     END IF
14: END FOR
15: IF customerType = "VIP"
16:     discount = 0.20
17: ELSE IF customerType = "Regular"
18:     discount = 0.10
19: ELSE IF customerType = "New"
20:     discount = 0.05
21: ELSE
22:     discount = 0
23: END IF
24: RETURN "Valid order, discount = " + discount
END FUNCTION
```

**TASK Complete the worksheet below for Snippets A, B, and C.**

| Snippet | Decision Points Found (P) | V(G) = P + 1 | Risk Classification |
|---|---|---|---|
| | | | |
| | | | |
| | | | |

***Bonus Challenge:*** *Modify Snippet C by adding two more independent decision points (e.g., an extra validation rule). Recalculate V(G) and confirm it now crosses into the "Moderate" band (11–20).*

## Activity 2.3 Halstead Metrics Calculation

Use Snippet A (CheckEven) above. Counting rule: treat each keyword/operator symbol (IF, MOD, =, PRINT, ELSE) as an operator, and each variable/literal (number, 2, 0, "Even", "Odd") as an operand. Ignore FUNCTION/END markers.

**TASK Count $n_1$, $n_2$, $N_1$, $N_2$, then calculate the five derived measures. Show your working.**

| Measure | Value / Formula | Working |
|---|---|---|
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

## Activity 2.4 Defect Density, MTBF, MTTR & DRE Case Study

### Case: Team Delta's Release Data

| Metric Input | Value |
|---|---|
| **Module size** | 8 KLOC |
| **Total defects found (all phases)** | 40 |
| **Defects found before release (in-house testing)** | 34 |
| **Defects found after release (by customers)** | 6 |
| **Total monitored production time (30 days)** | 720 hours |
| **Number of production failure incidents** | 4 |
| **Total downtime across those incidents** | 8 hours |

**TASK Calculate Defect Density, MTBF, MTTR, and Defect Removal Efficiency (DRE) from the data above. Show your formulas and working.**

| Metric | Formula Used | Calculation & Result |
|---|---|---|
| | | |
| | | |
| | | |
| | | |
| | | |

## Activity 2.5 DORA Metrics Case Study

### Case: Team Nova's Two-Week Deployment Log

| Day | Deployed? | Lead Time (hrs) | Caused Failure? | Recovery Time (hrs) |
|---|---|---|---|---|
| 1 | Yes | 5 | No | – |
| 2 | Yes | 3 | No | – |
| 3 | No | – | – | – |
| 4 | Yes | 8 | Yes | 2 |
| 5 | Yes | 4 | No | – |
| 6 | No | – | – | – |
| 7 | Yes | 6 | No | – |
| 8 | Yes | 2 | No | – |
| 9 | Yes | 10 | Yes | 4 |
| 10 | Yes | 5 | No | – |

Simplified benchmark table (adapted for classroom use):

| Metric | Elite | High | Medium | Low |
|---|---|---|---|---|
| Deployment Frequency | ≥ 1/day | 1/week – 1/day | 1/month – 1/week | < 1/month |
| Lead Time for Changes | < 1 day | 1 day – 1 week | 1 week – 1 month | > 1 month |
| Change Failure Rate | 0–5% | 6–10% | 11–15% | > 15% |
| MTTR | < 1 hour | < 1 day | < 1 week | > 1 week |

**TASK From the 10-day log, calculate all four DORA metrics and classify Team Nova's performance tier for each one using the benchmark table above.**

| Metric | Calculation | Result | Performance Tier |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

Reflection: Team Nova's results may look strong on some metrics and weak on others.

1. Which metric appears weakest, and what would you investigate first as the team's QA lead?
2. Propose one Rework Rate-style metric that could reveal a problem hidden by the four keys alone.

## Activity 2.6 (Reflection) Goodhart's Law Case Analysis

*Case: A manager told the team "increase your deployment frequency this quarter." The team responded by splitting single features into 15–20 trivial, near-meaningless deployments per week to inflate the number without any real increase in delivered value or user impact.*

**TASK (1) Explain why this is an example of Goodhart's Law. (2) Propose two balancing metrics or policy changes that would prevent this kind of gaming while still encouraging genuinely frequent, healthy delivery.**

Write your answer (150–250 words):

| Your Answer |
| --- |
| |
| |
| |
| |
| |
| |

## Practicum 2 Deliverables Checklist

- ☐ Activity 2.1 worksheet completed (UFP total calculated)
- ☐ Activity 2.2 worksheet completed (3 snippets + bonus challenge)
- ☐ Activity 2.3 worksheet completed (all 5 Halstead measures)
- ☐ Activity 2.4 worksheet completed (4 metrics calculated)
- ☐ Activity 2.5 worksheet completed (4 DORA metrics + reflection)
- ☐ Activity 2.6 written reflection completed

# Assessment Rubric

| Criterion | Weight | Excellent (90–100%) | Adequate (60–89%) | Needs Improvement (<60%) |
|---|---|---|---|---|
| Completeness | 25% | All activities fully attempted, including bonus items | Most activities attempted; minor gaps | Several activities missing or incomplete |
| Technical Accuracy | 35% | All calculations and analyses correct, with clear working shown | Mostly correct; a few calculation errors | Frequent errors or missing working |
| Analysis & Justification | 25% | Justifications specific, well-reasoned, and tied to concrete examples | Justifications present but generic | Little or no justification given |
| Clarity & Organization | 15% | Worksheets neatly completed and easy to follow | Mostly organized, minor clarity issues | Disorganized or difficult to follow |

# Quick-Reference Formula Sheet

## Complexity Metrics

| Metric | Formula |
|---|---|
| **Cyclomatic Complexity** | $V(G) = E - N + 2 \quad = \quad P + 1$ |
| **Halstead Program Length** | $N = N_1 + N_2$ |
| **Halstead Vocabulary** | $n = n_1 + n_2$ |
| **Halstead Volume** | $V = N \times \log_2(n)$ |
| **Halstead Difficulty** | $D = (n_1/2) \times (N_2/n_2)$ |
| **Halstead Effort** | $E = D \times V$ |

## Quality & Reliability Metrics

| Metric | Formula |
|---|---|
| **Defect Density** | Total Defects ÷ KLOC |
| **MTBF** | Total Uptime ÷ Number of Failures |
| **MTTR** | Total Downtime ÷ Number of Incidents |
| **Defect Removal Efficiency (DRE)** | (Defects Found Pre-Release ÷ Total Defects) × 100% |

## DORA Four Keys

| Metric | Formula / Definition |
|---|---|
| **Deployment Frequency** | Number of Deployments ÷ Time Period |
| **Lead Time for Changes** | Average(Time from Commit to Production) |
| **Change Failure Rate** | Failed Deployments ÷ Total Deployments × 100% |
| **Mean Time to Recovery** | Total Recovery Time ÷ Number of Incidents |

# Instructor Answer Key & Grading Guide

*This section is for instructor use only. Consider removing or redacting these pages before distributing the module to students.*

## Activity 1.1 Model Answer

A strong answer covers all 4 valid classes (D, C, B, A), the invalid classes (below 0, above 100), and every boundary. Example model set (13 cases):

| Score | Expected Output | Rationale |
|---|---|---|
| -1 | Invalid Input | Just below valid range |
| 0 | D | Lower boundary of valid range |
| 30 | D | Representative of class D |
| 59 | D | Upper boundary of D |
| 60 | C | Lower boundary of C |
| 68 | C | Representative of class C |
| 74 | C | Upper boundary of C |
| 75 | B | Lower boundary of B |
| 82 | B | Representative of class B |
| 89 | B | Upper boundary of B |
| 90 | A | Lower boundary of A |
| 100 | A | Upper boundary of valid range |
| 101 | Invalid Input | Just above valid range |

## Activity 1.2 Model Answer

Expected defects (students should find at least 4 of the following 5):

- D1 (Major) Line 3: loop runs “i = 0 TO size”, causing an out-of-bounds access on the last iteration; should be “TO size-1”.
- D2 (Major) Line 4: condition excludes zero values in addition to negatives, which is inconsistent with the stated intent of excluding only negative outliers.
- D3 (Minor) Line 10: returns 0 when count = 0, which is indistinguishable from a legitimate average of 0; should signal an error/undefined result instead.
- D4 (Minor) Line 12: sum / count risks integer-division truncation if both are integer types, losing precision.
- D5 (Minor) No validation for a null or empty input array.

## Activity 1.3 & 1.4 Grading Guide

Activity 1.3 has no single correct answer grade on justification quality per the rubric. Activity 1.4 model scenarios: Unit test the fine calculation function in isolation for a 5-day-late book; Integration test that Borrow Book correctly updates both the Catalog and Student Loan Record modules; System test a full borrow-to-return cycle end-to-end; Acceptance have a librarian confirm the report export matches the expected monthly format.

## Activity 2.1 Answer

Weights: Login=4, Add Student=4, Mark Attendance=3, Generate Report=7, Search Student=3, Student Master File=10, Course Master File=7, Payroll Interface=7. Total UFP = 4+4+3+7+3+10+7+7 = 45 function points.

## Activity 2.2 Answer

| Snippet | Decision Points (P) | V(G) = P + 1 | Risk |
|---|---|---|---|
| A | 1 (IF) | 2 | Simple |
| B | 4 (FOR, IF, ELSE IF, IF) | 5 | Simple |
| C | 8 (IF, FOR, 3×IF, IF, 2×ELSE IF) | 9 | Simple (near Moderate boundary) |

## Activity 2.3 Answer

$n_1 = 5$ {IF, MOD, =, PRINT, ELSE}; $n_2 = 5$ {number, 2, 0, “Even”, “Odd”}; $N_1 = 6$; $N_2 = 5$.

- Program Length: $N = 6 + 5 = 11$
- Vocabulary: $n = 5 + 5 = 10$
- Volume: $V = 11 \times \log_2(10) \approx 11 \times 3.32 \approx 36.5$
- Difficulty: $D = (5/2) \times (5/5) = 2.5$
- Effort: $E = 2.5 \times 36.5 \approx 91.3$

## Activity 2.4 Answer

- Defect Density = 40 ÷ 8 = 5 defects/KLOC
- MTBF = (720 − 8) ÷ 4 = 712 ÷ 4 = 178 hours
- MTTR = 8 ÷ 4 = 2 hours
- DRE = (34 ÷ 40) × 100% = 85%

## Activity 2.5 Answer

- Deployment Frequency = 8 deployments ÷ 10 days = 0.8/day (≈ 5–6/week) → High tier
- Lead Time for Changes = average of (5,3,8,4,6,2,10,5) = 43 ÷ 8 = 5.4 hours → Elite tier (< 1 day)
- Change Failure Rate = 2 ÷ 8 = 25% → Low tier (poor worth investigating)
- MTTR = (2 + 4) ÷ 2 = 3 hours → High tier (< 1 day, but not < 1 hour for Elite)

Discussion guide: Team Nova ships fast and often, but 1 in 4 deployments causes a failure a classic sign that speed has outpaced quality safeguards. A good answer should flag this tension and propose investigating root causes (e.g., insufficient pre-deployment testing) rather than simply celebrating the frequency numbers.

### Activity 2.6 Grading Guide

A strong answer identifies that deployment frequency became a target rather than a proxy for genuine delivery health, causing the team to optimize for the number itself (Goodhart's Law) rather than real user value. Acceptable balancing proposals include: pairing deployment frequency with a value/impact metric (e.g., features shipped that meet a minimum scope threshold), tracking Change Failure Rate and Rework Rate alongside frequency, or having a lead review deployments for meaningfulness rather than counting them automatically.